

Group Assignment 2

Naive Bayesian Anti-Spam Filtering

Replication and Cost-Sensitive Evaluation on the Ling-Spam Corpus

Course	CCS5600
Semester	Semester 2, 2025/2026
Group	[Insert group number]
Group members	[Insert names and student IDs]
Lecturer	[Insert lecturer name]
Submission date	[Insert submission date]

Draft note: replace bracketed placeholders before final PDF submission.

Abstract

This report implements and evaluates a Naive Bayes anti-spam classifier on the Ling-Spam corpus, following the core experimental design of Androutsopoulos et al. (2000). The corpus contains 2,893 emails, including 481 spam messages and 2,412 legitimate messages, organized into ten cross-validation folds and four preprocessing versions: Bare, Stop, Lemm, and Lemm_Stop. A text preprocessing pipeline was implemented to parse messages, extract labels, separate subject and body text, tokenize words, and construct vocabularies using the most frequent terms. The classifier was implemented from scratch using the Naive Bayes text-learning procedure described by Tom M. Mitchell, with Laplace smoothing and log probabilities. Performance was evaluated using spam precision, spam recall, weighted accuracy, and Total Cost Ratio under lambda values of 1, 9, and 99. Results show that larger vocabularies generally improve performance, while cost-sensitive evaluation reveals limitations hidden by accuracy alone. Stop-word removal produced the strongest TCR at $\lambda = 9$, while Lemm_Stop produced the strongest TCR at $\lambda = 99$.

1. Introduction

Spam filtering is a binary text classification problem in which incoming messages must be classified as either spam or legitimate email. Although ordinary classification accuracy is often used in machine learning, spam filtering has an asymmetric error structure. A false negative allows a spam message into the inbox, while a false positive blocks a legitimate message. In practical email systems, false positives are usually more harmful because the user may miss an important legitimate email.

This assignment replicates key ideas from the Ling-Spam experiments by Androutsopoulos et al. (2000). The main goals are to implement Naive Bayes text classification, evaluate preprocessing choices, investigate vocabulary size and training size, and apply cost-sensitive metrics that better reflect real spam-filtering risk.

2. Dataset and Preprocessing

The Ling-Spam corpus combines legitimate Linguist List messages with spam messages. Each corpus version is split into ten parts, which were used as predefined folds for ten-fold cross-validation. Files whose names begin with spmsg were labelled as spam, while all other files were labelled as ham.

Version	Description	Total	Spam	Ham
bare	No lemmatization, stop-words included	2893	481	2412
stop	No lemmatization, stop-words removed	2893	481	2412
lemm	Lemmatized, stop-words included	2893	481	2412
lemm_stop	Lemmatized, stop-words removed	2893	481	2412

Table 1. Ling-Spam corpus versions and class distribution.

The preprocessing script parsed each email by reading the first Subject line separately from the remaining body text. Subject and body were then combined for tokenization. Tokens were converted to lowercase alphabetic words. For each training fold, the vocabulary was built only from the training data to avoid data leakage into the test fold. Vocabulary sizes of $N = 50, 100, 500, 1000, 5000$, and all were evaluated.

3. Naive Bayes Classifier

The classifier was implemented from scratch using the Naive Bayes text-classification approach described by Mitchell. For each class v_j in {spam, ham}, the prior probability $P(v_j)$ was estimated as the number of training documents in that class divided by the total number of training documents.

For each vocabulary word w_k and class v_j , the conditional probability $P(w_k | v_j)$ was estimated using word occurrence counts from all training documents in that class. Laplace smoothing was applied so that unseen words did not receive zero probability: $P(w_k | v_j) = (\text{count}(w_k \text{ in class } v_j) + 1) / (\text{total word positions in class } v_j + \text{vocabulary size})$.

During classification, log probabilities were used instead of direct multiplication to avoid numerical underflow. A test email was assigned the class with the larger posterior log score before cost-sensitive thresholding was applied.

4. Cost-Sensitive Metrics

The evaluation used the confusion matrix terms TP, TN, FP, and FN. Spam precision measures how many predicted spam messages were truly spam, while spam recall measures how many true spam messages were detected. Weighted accuracy incorporates the cost factor lambda by assigning greater weight to legitimate messages when false positives are more costly.

The assignment's cost-sensitive decision rule classifies an email as spam only if $P(\text{Spam} | \text{Email}) / P(\text{Ham} | \text{Email}) > \lambda$. Because the implementation stores log scores, this was computed as $\text{spam_log_score} - \text{ham_log_score} > \log(\lambda)$. Total Cost Ratio compares the cost of using the filter to the cost of manually deleting all spam without a filter. A TCR above 1 indicates that the filter is beneficial.

5. Experiments and Results

5.1 Effect of Attribute-Set Size

The first experiment fixed the training size and varied the vocabulary size. The graphs below show the Lemm_Stop configuration for $N = 50, 100, 500, 1000, 5000$, and all. Three lambda values were evaluated: 1, 9, and 99.

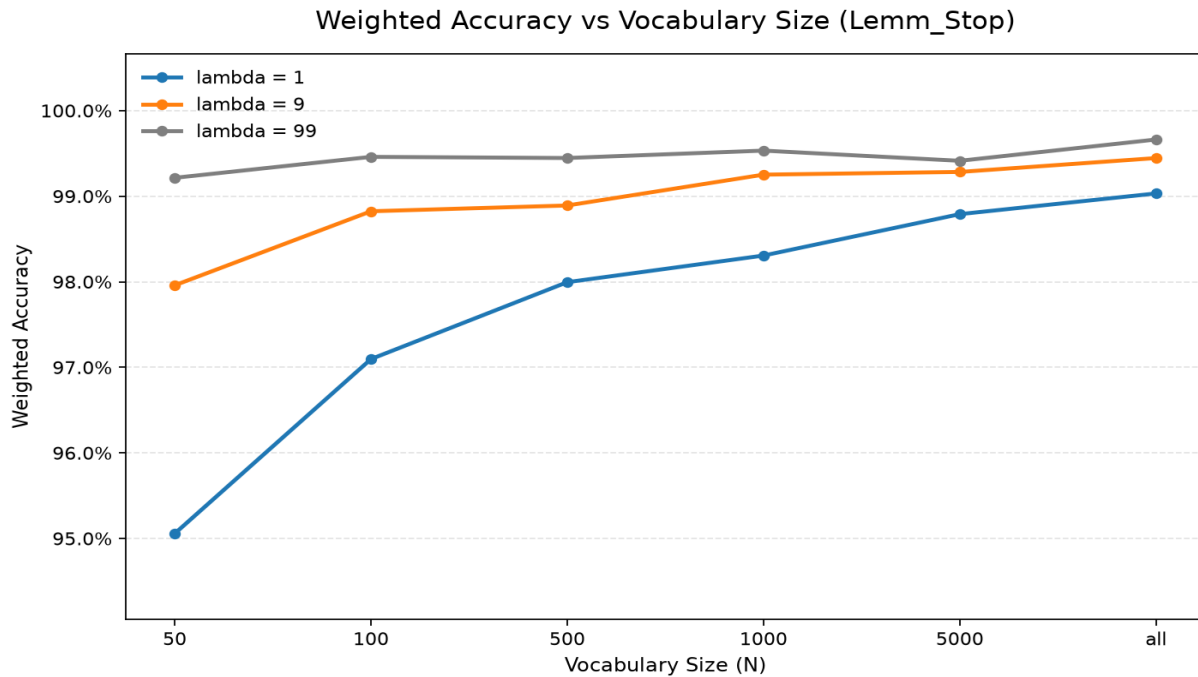


Figure 1. Weighted accuracy against vocabulary size for $\lambda = 1, 9$, and 99 using Lemm_Stop.

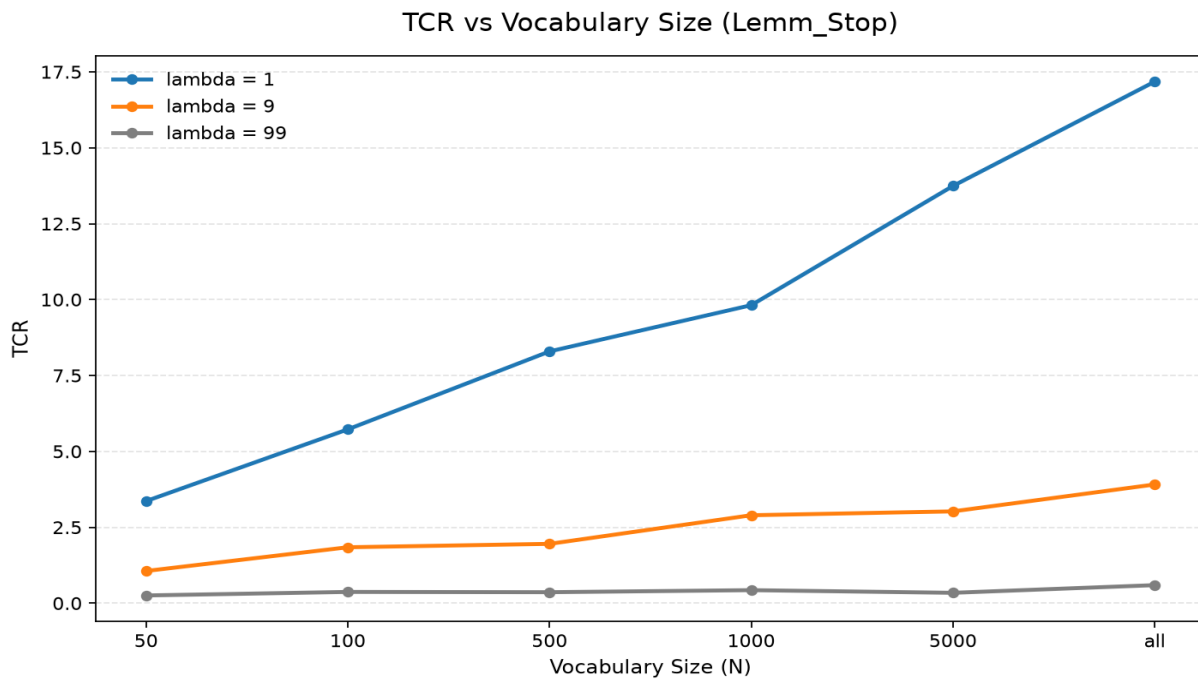


Figure 2. TCR against vocabulary size for $\lambda = 1, 9$, and 99 using Lemm_Stop.

Weighted accuracy remains high across most settings, especially for larger lambda values, because correctly classified ham messages receive high weight. TCR shows a sharper distinction between configurations and makes the cost of false positives more visible.

5.2 Effect of Preprocessing

The second experiment compared the four preprocessing versions using the full vocabulary. The Stop version achieved the highest TCR at $\lambda = 9$, while Lemm_Stop achieved the highest TCR

at $\lambda = 99$. This suggests that stop-word removal was especially useful for moderate cost sensitivity, while combining lemmatization and stop-word removal helped more under a stricter false-positive penalty.

Version	N	Precision	Recall	WAcc1	WAcc9	WAcc99	TCR9	TCR99
Bare	all	96.35%	98.75%	99.03%	99.24%	99.38%	2.86	0.32
Stop	all	97.71%	97.51%	99.20%	99.50%	99.58%	4.33	0.48
Lemm	all	96.49%	97.30%	99.00%	99.25%	99.29%	2.90	0.28
Lemm_Stop	all	97.49%	96.88%	99.03%	99.45%	99.66%	3.91	0.60

Table 2. Comparative performance across preprocessing versions using $N = \text{all}$.

5.3 Effect of Training-Corpus Size

The training-size experiment used the Stop configuration with $N = \text{all}$ because it produced the highest TCR at $\lambda = 9$ in the preprocessing comparison. The training portion of each fold was sampled at 10% increments from 10% to 100%, and the model was re-trained at each setting.

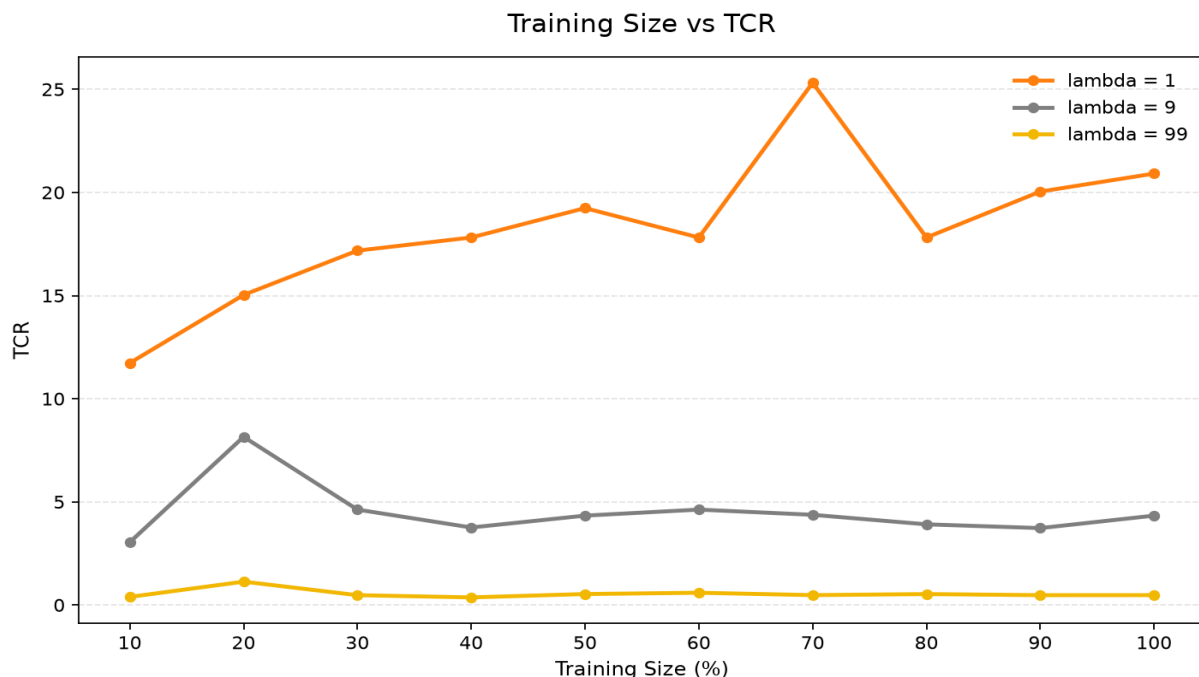


Figure 3. Learning curve showing TCR against training size for $\lambda = 1, 9$, and 99 using Stop with $N = \text{all}$.

The learning curve shows that the $\lambda = 1$ setting benefits most clearly from additional training data, with TCR generally increasing as training size grows. The $\lambda = 9$ curve remains beneficial but less stable, while $\lambda = 99$ remains below 1 for most settings, indicating that very high false-positive costs make the classifier difficult to justify without additional safeguards.

6. Discussion

The results show why accuracy alone is insufficient for spam filtering. Some configurations achieve very high weighted accuracy even when TCR is low. This occurs because the corpus contains many more ham messages than spam messages, and a classifier can appear accurate by correctly

classifying many ham messages. However, TCR directly penalizes false positives according to λ , so it better reflects the practical cost of wrongly blocking legitimate messages.

For example, in the preprocessing comparison, Lemm_Stop achieved 99.66% WAcc at $\lambda = 99$ but only 0.60 TCR. Since TCR is below 1, this configuration would not be beneficial under that strict cost model despite its very high weighted accuracy. This supports the reference paper's warning that Naive Bayes spam filters can require additional safety mechanisms when false positives are extremely costly.

Increasing vocabulary size generally improved both WAcc and TCR in the attribute-set experiment. Larger vocabularies allowed the classifier to capture more discriminative evidence. However, stronger performance at $\lambda = 1$ did not necessarily transfer to $\lambda = 99$, where a small number of false positives can sharply reduce TCR.

7. Conclusion

This assignment implemented a complete Naive Bayes anti-spam filtering pipeline for the Ling-Spam corpus. The pipeline parsed emails, generated vocabularies, trained a Mitchell-style Naive Bayes classifier, applied Laplace smoothing, and evaluated performance using ten-fold cross-validation. The strongest preprocessing result at $\lambda = 9$ was obtained by the Stop version with $N = \text{all}$, while Lemm_Stop performed best for TCR at $\lambda = 99$. The experiments demonstrate that cost-sensitive metrics are essential for spam-filter evaluation because high accuracy can hide unacceptable false-positive costs.

8. Reproducibility

All experiments and final figures were generated using Python scripts. The project folder used for this report is C:\Users\the50\Downloads\ML Assignment. The final Task 4 figures were generated from CSV outputs rather than manually edited in Excel.

Script	Purpose
task1_preprocess_all_versions.py	Parses corpus versions and creates preprocessing outputs.
task2_naive_bayes.py	Implements Mitchell-style Naive Bayes with Laplace smoothing.
task3_metrics.py	Applies cost-sensitive decisions and computes WAcc, precision, recall, and TCR.
task4_training_size_experiment.py	Runs the 10% to 100% training-size experiment.
task4_generate_final_outputs.py	Generates final figures and comparison table from CSV results.

Table 3. Python scripts used for reproducibility.

References

Androutsopoulos, I., Koutsias, J., Chandrinou, K. V., Paliouras, G., & Spyropoulos, C. D. (2000). An Evaluation of Naive Bayesian Anti-Spam Filtering. Proceedings of the Workshop on Machine Learning in the New Information Age, 11th European Conference on Machine Learning, Barcelona, Spain, 9-17.

Mitchell, T. M. (1997). Machine Learning. McGraw-Hill. Naive Bayes text-classification algorithm referenced in the assignment brief.

Ling-Spam corpus README. The dataset contains four versions of the corpus and ten predefined folds for cross-validation.